

maix.protocol

maix.protocol module

You can use `maix.protocol` to access this module with MaixPy
This module is generated from `MaixPy` and `MaixCDK`

1. Module

No module

2. Enum

2.1. CMD

protocol cmd, more doc see MaixCDK document's convention doc

item	describe
note	max app custom CMD value should < CMD_APP_MAX
values	CMD_APP_MAX : 200, max app custom CMD value should < CMD_APP_MAX CMD_SET_REPORT : set auto upload data mode

item	describe
	CMD_APP_LIST: CMD_START_APP: CMD_EXIT_APP: CMD_CUR_APP_INFO: CMD_APP_INFO: CMD_KEY: CMD_TOUCH:

C++ defination code:

```

1  enum CMD
2      {
3          CMD_APP_MAX = 0xC8,
4          // 200, max app custom CMD value
5          should < CMD_APP_MAX
6
7          CMD_SET_REPORT =
8          0xF8, // set auto upload data
9          mode
10         CMD_APP_LIST =
11         0xF9,
12         CMD_START_APP =
13         0xFA,
14         CMD_EXIT_APP =
15         0xFB,
16         CMD_CUR_APP_INFO =
17         0xFC,
18         CMD_APP_INFO =
19         0xFD,
20         CMD_KEY =
21         0xFE,
22         CMD_TOUCH =
23         0xFF,
24     }

```

2.2. FLAGS

protocol flags, more doc see MaixCDK document's convention doc

item	describe
values	FLAG_REQ: FLAG_RESP: FLAG_IS_RESP_MASK: FLAG_RESP_OK: FLAG_RESP_ERR: FLAG_RESP_OK_MASK: FLAG_REPORT: FLAG_REPORT_MASK: FLAG_VERSION_MASK:

C++ defination code:

```
1  enum FLAGS
2      {
3          FLAG_REQ = 0x00,
4          FLAG_RESP = 0x80,
5          FLAG_IS_RESP_MASK =
6      0x80,
7
8          FLAG_RESP_OK = 0x40,
9          FLAG_RESP_ERR = 0x00,
10         FLAG_RESP_OK_MASK =
11     0x40,
12
13         FLAG_REPORT = 0x20,
14         FLAG_REPORT_MASK =
15     0x20,
16
17         FLAG_VERSION_MASK =
18     0x03
19     }
```

3. Variable

3.1. VERSION

protocol version

item	description
value	1
readonly	True

C++ definition code:

```
1 | const uint8_t VERSION = 1
```

3.2. HEADER

protocol header

item	description
readonly	False

C++ definition code:

```
1 | extern uint32_t HEADER
```

4. Function

4.1. crc16_IBM

```
1 | def crc16_IBM(data:  
    maix.Bytes(bytes)) -> int
```

item	description
param	data: data, bytes type.
return	CRC16-IBM value, uint16_t type.

C++ definition code:

```
1 | uint16_t crc16_IBM(const Bytes  
   | *data)
```

5. Class

5.1. MSG

protocol msg

C++ definition code:

```
1 | class MSG
```

5.1.1. version

protocol version

item	description
type	var
static	False
readonly	False

C++ definition code:

```
1 | uint8_t version
```

5.1.2. resp_ok

Indicate response message type, true means CMD valid and the CMD processed correctly, (only for response msg)

item	description
type	var
static	False
readonly	False

C++ defination code:

```
1 | uint8_t resp_ok
```

5.1.3. has_been_replied

Flag whether CMD has been processed and responded to CMD sender.\nE.g. CMD CMD_START_APP will be automatically processed in CommProtocol.get_msg function,\nso the return msg will set this flag to true.

item	description
type	var
static	False
readonly	False

C++ definition code:

```
1 | bool has_been_replied{false}
```

5.1.4. cmd

CMD value

item	description
type	var
static	False
readonly	False

C++ definition code:

```
1 | uint8_t cmd
```

5.1.5. is_resp

message is response or not, contrast with is_req

item	description
type	var
static	False
readonly	False

C++ definition code:

```
1 | bool is_resp
```

5.1.6. is_req

message is request or not, contrast with is_resp

item	description
type	var
static	False
readonly	False

C++ definition code:

```
1 | bool is_req
```

5.1.7. is_report

message is request or not, contrast with is_resp

item	description
type	var
static	False
readonly	False

C++ definition code:

```
1 | bool is_report
```

5.1.8. body_len

Message body length, read only, use set_body() to update

item	description
type	var
attention	DO NOT manually change this value
static	False
readonly	False

C++ definition code:

```
1 | int body_len
```

5.1.9. encode_resp_ok

```
1 | def encode_resp_ok(*args, **kwargs)
```

Encode response ok(success) message

item	description
type	func
param	body : response body, can be null
return	encoded data, if nullptr, means error, and the error code is -err.Err
static	False

C++ definition code:

```
1 | Bytes *encode_resp_ok(Bytes *body
    = nullptr)
```

5.1.10. encode_report

```
1 | def encode_report(*args, **kwargs)
```

Encode proactively report message

item	description
type	func
param	body : report body, can be null
return	encoded data, if nullptr, means error, and the error code is -err.Err
static	False

C++ definition code:

```
1 | Bytes *encode_report(Bytes *body  
   | = nullptr)
```

5.1.11. encode_resp_err

```
1 | def encode_resp_err(*args, **kwargs)
```

Encode response error message

item	description
type	func
param	code : error code msg : error message

item	description
return	encoded data, if nullptr, means error, and the error code is -err.Err
static	False

C++ defination code:

```
1 | Bytes *encode_resp_err(err::Err
    code, const std::string &msg)
```

5.1.12. set_body

```
1 | def set_body(self, body_new:
    maix.Bytes(bytes)) -> None
```

Update message body

item	description
type	func
param	body_new : new body data
static	False

C++ defination code:

```
1 | void set_body(Bytes *body_new)
```

5.1.13. get_body

```
1 | def get_body(*args, **kwargs)
```

Get message body

item	description
type	func
return	message body, bytes type
static	False

C++ definition code:

```
1 | Bytes *get_body()
```

5.2. Protocol

Communicate protocol

C++ definition code:

```
1 | class Protocol
```

5.2.1. __init__

```
1 | def __init__(self, buff_size: int =  
    1024, header: int = 3148663466) ->  
    None
```

Construct a new Protocol object

item	description
type	func
param	buff_size : Data queue buffer size

item	description
static	False

C++ definition code:

```
1 | Protocol(int buff_size = 1024,
    | uint32_t
    | header=maix::protocol::HEADER)
```

5.2.2. buff_size

```
1 | def buff_size(self) -> int
```

Data queue buffer size

item	description
type	func
static	False

C++ definition code:

```
1 | int buff_size()
```

5.2.3. push_data

```
1 | def push_data(self, new_data:
    | maix.Bytes(bytes)) -> maix.err.Err
```

Add data to data queue

item	description
type	func
param	new_data: new data
return	error code, maybe err.Err.ERR_BUFF_FULL
static	False

C++ definition code:

```
1 | err::Err push_data(const Bytes
   | *new_data)
```

5.2.4. decode

```
1 | def decode(self, new_data:
   | maix.Bytes(bytes) = None) -> MSG
```

Decode data in data queue and return a message

item	description
type	func
param	new_data: new data add to data queue, if null, only decode.
return	decoded message, if nullptr, means no message decoded.
static	False

C++ definition code:

```
1 | protocol::MSG *decode(const Bytes
    *new_data = nullptr)
```

5.2.5. encode_resp_ok

```
1 | def encode_resp_ok(*args, **kwargs)
```

Encode response ok(success) message to buffer

item	description
type	func
param	cmd : CMD value body : response body, can be null
return	encoded data, if nullptr, means error, and the error code is -err.Err
static	False

C++ definition code:

```
1 | Bytes *encode_resp_ok(uint8_t
    cmd, Bytes *body = nullptr)
```

5.2.6. encode_report

```
1 | def encode_report(*args, **kwargs)
```

Encode proactively report message to buffer

item	description
type	func

item	description
param	cmd : CMD value body : report body, can be null
return	encoded data, if nullptr, means error, and the error code is -err.Err
static	False

C++ definition code:

```
1 | Bytes *encode_report(uint8_t cmd,
   | Bytes *body = nullptr)
```

5.2.7. encode_resp_err

```
1 | def encode_resp_err(*args, **kwargs)
```

Encode response error message to buffer

item	description
type	func
param	cmd : CMD value code : error code msg : error message
return	encoded data, if nullptr, means error, and the error code is -err.Err
static	False

C++ definition code:


```
1 | Bytes *encode_resp_err(uint8_t  
   cmd, err::Err code, const  
   std::string &msg)
```

